Java Backend Architecture

Interview Series

Chapter 16.10 Debugging & Troubleshooting Containers

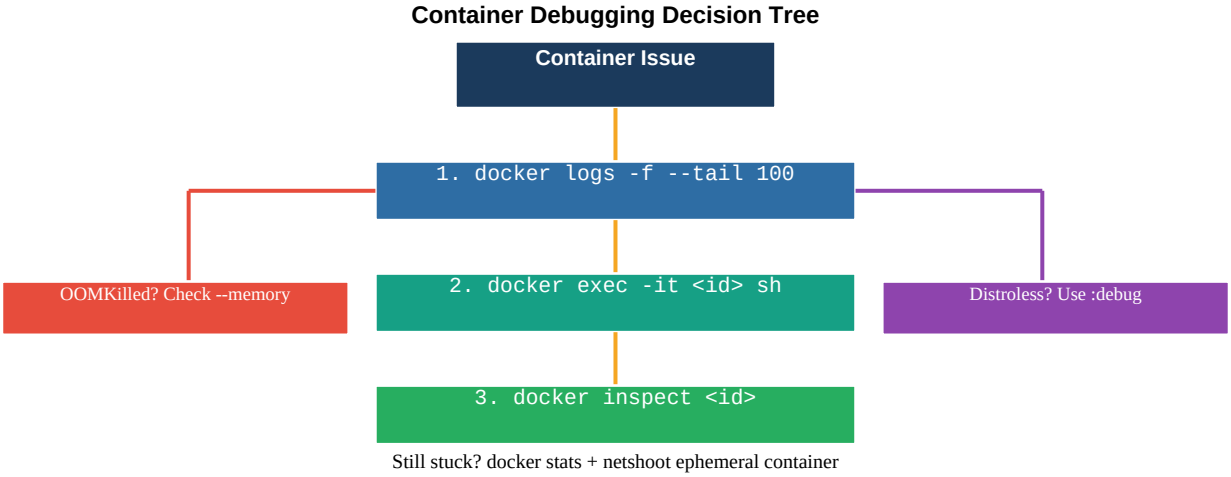
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 16.10 – Debugging & Troubleshooting Containers

Container Debugging Overview

Effective container debugging requires a systematic approach. Start with logs, escalate to exec-based inspection, use docker inspect for configuration issues, and employ ephemeral debug containers for distroless or restricted environments.

Debugging Decision Tree



docker logs

```
# Follow logs with tail docker logs -f --tail 100 mycontainer # Show logs since timestamp docker logs --since "2024-01-15T10:00:00" mycontainer # Show logs with timestamps docker logs -t mycontainer # Show logs for last 30 minutes docker logs --since 30m mycontainer # Grep for errors docker logs mycontainer 2>&1 | grep -i "error\|exception\|warn" # Save logs to file docker logs mycontainer > container.log 2>&1
```

docker exec – Interactive Shell

```
# Open interactive shell docker exec -it mycontainer sh docker exec -it mycontainer bash # if bash available # Run single command docker exec mycontainer ps aux docker exec mycontainer cat /etc/resolv.conf # Check Java process docker exec mycontainer jps -lv docker exec mycontainer jstack <pid> # thread dump docker exec mycontainer jmap -heap <pid> # heap summary # Check environment variables docker exec mycontainer env | sort # Check file permissions docker exec mycontainer ls -la /app/
```

docker inspect

```
# Full JSON inspection docker inspect mycontainer # Extract specific fields with Go templates docker inspect --format="{{.State.Status}}" mycontainer docker inspect --format="{{.State.OOMKilled}}" mycontainer docker inspect --format="{{.State.ExitCode}}" mycontainer docker inspect --format="{{.NetworkSettings.IPAddress}}" mycontainer # Health status docker inspect --format="{{json .State.Health}}" mycontainer | jq . # Environment variables docker inspect --format="{{json .Config.Env}}" mycontainer | jq . # Mounts docker inspect --format="{{json .Mounts}}" mycontainer | jq .
```

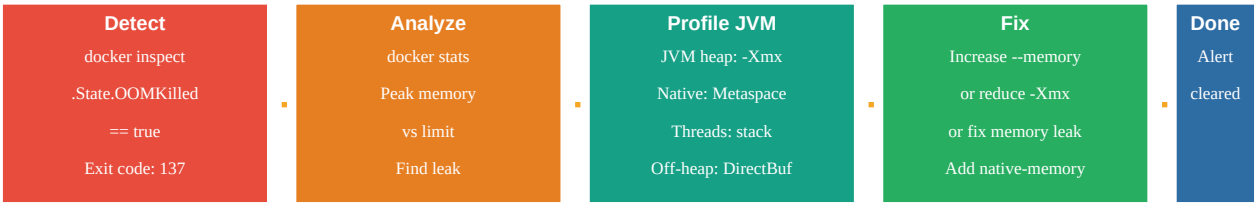
docker stats – Resource Monitoring

```
# Live resource stats for all containers docker stats # Stats for specific container (no-stream = snapshot) docker stats --no-stream mycontainer # Output: CONTAINER CPU% MEM USAGE/LIMIT MEM%
```

```
NET I/O BLOCK I/O PIDS # Example: myapp 45.2% 450MiB/512MiB 88% 1.2GB/890MB 0B/0B 42 #
Formatted output docker stats --format "table {{.Name}} {{.CPUPerc}} {{.MemUsage}}
{{.MemPerc}}"
```

OOMKilled Diagnosis

OOMKilled Diagnosis Workflow

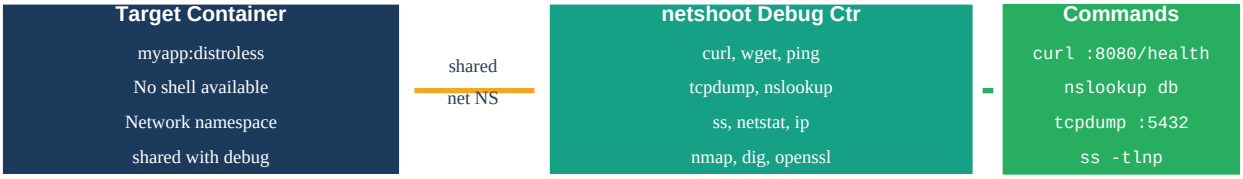


Total container memory = JVM heap + Metaspace + threads + DirectBuffers + native libs

```
# Detect OOMKill docker inspect mycontainer | jq ".[0].State.OOMKilled" # true docker inspect
mycontainer | jq ".[0].State.ExitCode" # 137 # JVM memory breakdown for containers # Total =
heap (-Xmx) + Metaspace + CompressedClassSpace # + Stack (threads x -Xss) + CodeCache +
DirectBuffers # Rule: --memory = Xmx + 25-30% overhead # Example: -Xmx400m + set --memory 512m
# Use JVM flags for container awareness JAVA_OPTS="-XX:+UseContainerSupport \
-XX:MaxRAMPercentage=75.0 \ -XX:+HeapDumpOnOutOfMemoryError \
-XX:HeapDumpPath=/tmp/heapdump.hprof"
```

netshoot – Ephemeral Debug Container

netshoot Ephemeral Debug Container



`docker run --rm -it --network container:<id> nicolaka/netshoot`

```
# Attach to running container network namespace docker run --rm -it --network container:myapp
nicolaka/netshoot # Debug DNS issues nslookup postgres # resolve service name dig postgres A #
detailed DNS query cat /etc/resolv.conf # check DNS config # Debug connectivity curl -v
http://postgres:5432 # test TCP connect nc -zv redis 6379 # netcat port check tcpdump -i eth0
port 5432 # capture DB traffic # On host network (debug docker0) docker run --rm -it --network
host nicolaka/netshoot bridge fdb show # MAC address table
```

Healthcheck Debugging

```
# Check healthcheck status docker inspect --format="{{json .State.Health}}" mycontainer | jq .
# Output: # { # "Status": "unhealthy", # "FailingStreak": 3, # "Log": [{ # "Start":
"2024-01-15T10:00:00Z", # "End": "2024-01-15T10:00:10Z", # "ExitCode": 1, # "Output": "curl:
(7) Failed to connect to localhost port 8080" # }] # } # Common causes: # - Spring Boot still
starting: increase start_period # - No curl in distroless: use wget or Java healthcheck # -
Wrong port: verify EXPOSE port matches actual port
```

Container Exit Codes

Exit Code	Meaning	Common Cause & Action
0	Success / Normal exit	App exited normally (batch job complete)
1	Application error	App threw uncaught exception; check logs

2	Shell built-in misuse	Shell syntax error in CMD/ENTRYPOINT
126	Command not executable	Permission denied on ENTRYPOINT binary
127	Command not found	Binary missing; check PATH and image contents
128+N	Fatal signal N	Killed by signal N
130	SIGINT (Ctrl+C)	Manually interrupted
137	SIGKILL / OOMKilled	docker kill or memory limit exceeded
143	SIGTERM	docker stop; graceful shutdown triggered
255	Exit status out of range	SSH or exec error; check exec command

Common Issues & Solutions

Port already in use	docker: Error bind: address already in use. Fix: ss -tlnp | grep 8080 to find process; docker ps to check other containers using the port; change host port: -p 8081:8080.
Volume permissions	Container writes fail with EACCES. Fix: COPY --chown=user:group or RUN chown in Dockerfile. Check: docker exec container ls -la /data. For named volumes: volume may be owned by root if created before USER switch.
DNS failures	java.net.UnknownHostException: postgres. Fix: verify containers on same user-defined network (not default bridge); docker network inspect; check /etc/resolv.conf in container.
Image pull errors	unauthorized: authentication required. Fix: docker login to registry; verify image name/tag exists; check rate limits (docker.io 100 pulls/6h anonymous).
Distroless debugging	No shell for docker exec. Fix: use :debug variant (has busybox sh); or use ephemeral debug container: docker run --pid=container:myapp gcr.io/distroless/java21-debian12:debug sh.
Java app OOMKilled	Container memory limit too low for JVM. Fix: set -XX:+UseContainerSupport -XX:MaxRAMPercentage=75 so JVM respects container limits; ensure --memory > Xmx + overhead.

Interview Q&A; – Debugging & Troubleshooting Containers

Q1. What is your systematic approach to debugging a failing container?

Systematic approach: (1) docker logs --tail 100 -f — check application logs for errors, exceptions, startup failures. (2) docker inspect -- check State (Running, OOMKilled, ExitCode), Config (Env vars, Cmd), Mounts, NetworkSettings. (3) docker stats — CPU/memory usage patterns (is it hitting limits?). (4) docker exec -it sh — interactive shell to check files, network, process state. (5) Ephemeral debug container (netshoot) for network issues. (6) docker top — see processes inside container. (7) Check healthcheck logs via inspect. Start with logs — 80% of issues are there.

Q2. What does exit code 137 mean and how do you diagnose it?

Exit code 137 = container received SIGKILL (signal 9). Two causes: (1) Someone ran docker kill (or a timeout expired in docker stop). (2) OOMKilled — kernel killed the process because the container exceeded its memory limit. Check: docker inspect --format='{{.State.OOMKilled}}' container returns true if OOM. Also check: docker events --filter container=myapp for kill events. Fix for OOM: increase --memory limit, reduce JVM heap (-Xmx), use -XX:MaxRAMPercentage=75 for container-aware heap sizing, fix memory leak.

Q3. How does docker inspect help with debugging?

docker inspect returns a comprehensive JSON document covering everything Docker knows about a container. Key debug fields: State (Running, Paused, Restarting, OOMKilled, ExitCode, Error, StartedAt, FinishedAt), Config (Env — all environment variables, Cmd, Entrypoint, Image), HostConfig (Memory limit, CpuQuota, Binds/mounts, PortBindings, SecurityOpt), NetworkSettings (IPAddress, Ports, Networks), Mounts (type, source, destination, mode), State.Health.Log (healthcheck output history). Use --format with Go templates for targeted extraction.

Q4. What is docker stats and what metrics does it provide?

docker stats provides live resource usage per container: CPU% (percentage of host CPU), MEM USAGE/LIMIT (current memory use vs limit), MEM% (memory percentage), NET I/O (bytes sent/received over container network), BLOCK I/O (bytes read/written to container filesystem/volumes), PIDS (number of processes in the container). Use --no-stream for a one-time snapshot. Critical thresholds: MEM% > 80% = risk of OOMKill; CPU% consistently 100% = CPU-bound or CPU limit too low; PIDS approaching limit = potential fork bomb.

Q5. What is OOMKilled and how do you diagnose it in Java containers?

OOMKilled means the Linux kernel's OOM Killer killed a process in the container because the container exceeded its memory limit (--memory flag). For Java: JVM memory is NOT just heap. Total memory = heap (Xmx) + Metaspace + CompressedClassSpace + Thread stacks (threads × Xss) + Code Cache + Direct Buffers + native libraries. Rule of thumb: set --memory to Xmx × 1.3. Better: use -XX:+UseContainerSupport -XX:MaxRAMPercentage=75 — JVM automatically sets heap to 75% of container memory. For analysis: -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/ for heap analysis.

Q6. How do you debug a distroless container that has no shell?

Options: (1) Use the :debug variant: gcr.io/distroless/java21-debian12:debug has busybox with sh — docker exec -it myapp sh works. (2) Ephemeral debug container sharing namespaces: docker run --rm -it --pid=container:myapp --network=container:myapp ubuntu sh — shares PID and network namespace. (3) Kubernetes ephemeral containers: kubectl debug -it mypod --image=ubuntu --target=myapp. (4) Add diagnostic tools in a dev build variant (multi-stage with debug tools in separate Dockerfile.debug). (5) Pre-flight checks: log all diagnostics at startup.

Q7. How do you investigate container network connectivity issues?

Step-by-step: (1) docker exec cat /etc/resolv.conf — verify DNS server (127.0.0.11 for user-defined networks). (2) docker exec nslookup service-name — test DNS resolution. (3) docker exec curl -v http://service:port — test TCP connectivity. (4) docker network inspect mynet — check which containers are connected and their IPs. (5) Attach netshoot: docker run --rm -it --network container:myapp nicolaka/netshoot. (6) tcpdump -i eth0 inside netshoot to capture traffic. (7) iptables -L DOCKER -n on host to check forwarding rules.

Q8. What is nicolaka/netshoot and what tools does it include?

nicolaka/netshoot is a Docker/Kubernetes network troubleshooting image containing ~100 networking tools. Key tools: curl/wget (HTTP testing), ping/traceroute (ICMP), nslookup/dig (DNS), ss/netstat (socket stats), tcpdump/tshark (packet capture), nmap (port scanning), iperf (bandwidth testing), ip/route (routing), nc/socat (TCP utilities), openssl (TLS testing), strace (system call tracing), jq (JSON parsing), and more. Attach to running container's network namespace: docker run --rm -it --network container:TARGET_ID nicolaka/netshoot.

Q9. What are common Java application startup failures in containers?

Common causes: (1) Missing environment variables — Spring Boot fails with IllegalArgumentException. Check: docker exec env. (2) Database not ready — Spring Boot tries to connect before DB healthcheck passes. Fix: depends_on condition: service_healthy or implement retry in app. (3) Port binding failure — another container on same port. Fix: docker ps -a to see all containers. (4) JVM crash — OOMKilled during startup. Fix: increase memory, reduce heap size. (5) Class not found — wrong JAR packaging. Fix: java -jar target/app.jar locally to verify. (6) Missing config file — volume mount incorrect. Fix: docker inspect Mounts.

Q10. How do you monitor container health over time?

Docker healthcheck provides point-in-time status. For continuous monitoring: (1) Prometheus + cAdvisor: scrapes container CPU, memory, network metrics; Grafana for dashboards. (2) docker stats --format in a script for basic monitoring. (3) Cloud-native: CloudWatch Container Insights (ECS), GKE metrics, Azure Monitor. (4) docker events --filter type=container monitors container lifecycle events (start, stop, die, OOMKill). (5) ELK/Loki for log aggregation and alerting on error patterns. Set up alerts on: OOMKilled events, container restarts > 3, CPU > 90% sustained, memory > 85%.

Q11. What does docker top show and when is it useful?

docker top displays the running processes inside a container (equivalent to ps inside the container). Shows: PID (container PID), UID, CMD (command and arguments), and other ps columns. Useful for: (1) Verifying the correct process is running as PID 1. (2) Checking for unexpected processes (indicates container compromise or misconfiguration). (3) Finding the PID of a Java process for jstack/jmap operations. (4) Checking thread count for Java apps. (5) Verifying non-root user (UID column). For distroless containers: docker top still works even without ps inside the container.

Q12. How do you perform a thread dump of a Java process in a container?

Methods: (1) docker exec myapp jstack — requires JDK in container (JRE-only won't have jstack). (2) Send SIGQUIT signal: docker kill --signal=QUIT myapp — JVM prints thread dump to stdout/stderr (visible in docker logs). (3) docker exec myapp kill -3 — same as SIGQUIT. (4) Attach JVisualVM remotely: expose JMX port (-Dcom.sun.management.jmxremote.port=9010) and connect via docker run -p 9010:9010. (5) Spring Boot Actuator: GET /actuator/threaddump (requires spring-boot-starter-actuator). Prefer SIGQUIT method for distroless containers.

Q13. What are port conflict errors and how do you resolve them?

Error: docker: Error response from daemon: driver failed programming external connectivity: Bind for 0.0.0.0:8080 failed: port is already allocated. Causes: another container already uses host port 8080; a host

process bound to 8080. Resolution: (1) docker ps -a to find container using 8080. (2) ss -tlnp | grep 8080 or netstat -tlnp | grep 8080 to find host process. (3) Stop conflicting container: docker stop . (4) Use different host port: -p 8081:8080. (5) Use an internal port only (no -p) and proxy via nginx. In CI/CD: use random port mapping (-p 8080) to avoid conflicts.

Q14. How do you debug volume permission issues?

Symptom: java.io.IOException: Permission denied writing to /app/logs. Debug: (1) docker exec container ls -la /app/logs — check ownership and permissions. (2) docker inspect Mounts — verify source path and options (read-only?). (3) id inside container — check current user/group. Root causes: (a) Named volume created before USER instruction — owned by root. Fix: chown in entrypoint or Dockerfile before USER. (b) Host bind mount owned by different user. Fix: match UID in container USER to host file owner. (c) Read-only volume. Fix: change :ro to :rw or use separate writable path.

Q15. What is the difference between docker events and docker logs?

docker logs shows stdout/stderr output from the main container process (application logs). docker events shows Docker daemon lifecycle events: container start, stop, die, OOMKill, exec_create, exec_start, network connect, volume mount, image pull. Use docker events to: detect unexpected container restarts (die + start loop = crash loop), audit container activity, detect OOMKill events in real time, monitor container scaling events. Filter: docker events --filter type=container --filter event=die. Combine with --format for structured output. Unlike logs, events are daemon-level, not application-level.

Q16. How do you debug an application that fails only in production containers but works locally?

Systematic approach: (1) Compare environments: docker inspect env vs local env — different config? Missing secrets? (2) Check resource limits: docker stats — is container hitting CPU/memory limits? (3) Check image version: ensure same image tag in prod and dev. Use digest pinning. (4) Check volume mounts: prod may have different config files bound in. (5) Network differences: user-defined network vs host? DNS resolving differently? (6) Race conditions: depends_on not waiting for readiness in prod (larger system startup time). (7) Log level: ensure DEBUG/INFO logging in prod to capture failure. (8) Reproduce with same resource limits locally.